NucleScript: Effects, Confirmations and Channel-Typed Pools for DNA Data Storage Programs

Vyom Kulshrestha
Independent Researcher, India
ORCID 0009-0009-1434-7148

Technical report, 8 November 2026
NucleScript 1.0, NucleOS v0.1.70

Abstract

Storing data in DNA means paying for synthesis and sequencing, destroying material when data is deleted, and reading it back through noisy channels whose error rates depend on the instrument. NucleScript is a small language for writing such storage programs. A pool's type names the sequencer profile it was read through and an error rate that consensus lowers. Operations that synthesise, sequence or delete have effects, each needs a confirmation written at the operation, and a function type can bound the effects of the closures passed through it. NucleScript is implemented in Rust as part of NucleOS, which supplies the codecs (ternary, DNA Fountain, Yin-Yang, HEDGES), Reed–Solomon parity, a noise simulator with channels measured from real oligo pools, and consensus and clustering of real reads. The checker is checked in two ways. A runtime monitor runs the shipped examples and generated programs and compares what each did with what the checker promised; with the work of writing the rules down formally, it has found eight defects, all fixed. A Lean 4 proof shows, for a core calculus with closures, aliases, closures the checker cannot see into, and higher-order functions, that an accepted program performs only confirmed operations, only effects the checker reports, and nothing a closure's ceiling forbids. On public data, a NucleScript program decodes 99.73% and 96.94% of the strands of two Illumina oligo pools with their labels, and 99.73% and 95.82% without them; the same decoder recovers 94.7% of 2,000 MinION clusters. No NucleScript program has yet driven a real synthesis or sequencing run.

1 Introduction

DNA data storage writes a file by synthesising short oligonucleotides that encode it, and reads the file back by sequencing them [6, 13, 10, 19]. Each write costs money and laboratory time. Each read costs more and returns many noisy, unordered copies of each strand, so a decoder has to cluster them, take a consensus, and fill the gaps with an outer code. Deleting data means destroying physical material. The channel is very different across instruments: Illumina reads miss about one base in a thousand, while nanopore reads get several percent wrong.

The software around this is a set of libraries and simulators [21, 24, 5, 25], driven by scripts. Nothing checks such a script before it runs. A script can delete a pool's contents without anyone confirming it, store data at a redundancy that the chosen sequencer cannot recover from, or ask for a retention period the storage medium cannot reach. In ordinary programs such mistakes cost a rerun. Here they cost a synthesis order or a pool of molecules.

NucleScript makes these properties part of a program's types. This report describes the language and its implementation, and then asks how far its checker can be trusted. Its contributions are:

- A language design (Sections 3 and 4): pools typed by the channel they were read through, effects for synthesis, sequencing and deletion whose confirmations are written at the operation

and checked at compile time, and effect ceilings on function types that bound what a closure passed through them may do.

- An implementation inside NucleOS (Section 5), with four codecs, Reed–Solomon parity, channels measured from real oligo pools, decoding of real reads, an editor extension and a stability promise for language version 1.0.
- A runtime monitor and a random program generator that check the checker's promises on running programs (Section 6). Together with formalising the rules, they found eight defects in the checker, all now fixed.
- A proof in Lean 4 [9] that the effect and confirmation rules keep three of the checker's four promises for a core calculus of the language (Section 7).
- An evaluation of decoding on public sequencing data (Section 8): a MinION dataset and two Illumina oligo pools, the latter decoded from inside a NucleScript program.

The limits come first: no NucleScript program has driven a real synthesizer or sequencer, and the decoding results use datasets other groups produced. Section 10 lists the rest.

2 Background

Writing. Oligo pools are synthesised in parallel on arrays, typically 100 to 350 nucleotides per strand. Synthesis is cheaper and more accurate when the GC content stays near half and runs of one base (homopolymers) are short. It introduces substitutions, insertions and deletions, and some strands are lost altogether. A codec maps bits to bases while respecting these constraints: the rotating ternary code of Goldman et al. [13], DNA Fountain's screened LT code [10], the Yin-Yang code [20], and HEDGES [22], which corrects insertions and deletions directly.

Reading. A sequencer returns reads in no particular order, several per strand. Coverage varies from strand to strand, roughly log-normally, and some strands get no reads. Illumina reads have mostly substitutions; nanopore reads have substitutions, insertions and deletions at a few percent each. Decoding clusters the reads by strand [23], takes a consensus per cluster [26], and lets an outer erasure code such as Reed–Solomon fill in strands that are missing or wrong.

Keeping. DNA decays. Strand breaks follow Arrhenius kinetics, measured for DNA in bone [1], in silica [14] and in dry anoxic capsules [8], so whether a file is still readable after a given time depends on the medium, the temperature and the strand length.

3 The language by example

A NucleScript program declares pools and runs operations on them. Listing 1 declares a pool written with the Yin-Yang codec at 4× redundancy and read on Illumina.

```
pool medical_archive: DnaPool {
    codec: YinYang,
    redundancy: 4x,
    profile: Illumina
}

let noisy: Pool<Illumina, 0.35%> = simulate medical_archive under Illumina
let recovered: Pool<Recovered> = consensus_vote(noisy, coverage: 10x)
```

```
pipeline archive_patient_records {
    encode "patient_records_2026.csv" using YinYang,
    protect with redundancy 4x,
    store into medical_archive,
    verify roundtrip
}
```

Listing 1: A pool, a simulated read, a consensus and a pipeline (`hero.nsl`).

The pool declaration is checked at compile time against the constraints of Section 2. The type `Pool<Illumina, 0.35%>` says the simulated read went through the Illumina profile and carries its error rate; `consensus_vote` at 10× coverage gives a recovered pool with a lower rate. The pipeline encodes a real file, adds parity strands, stores them in the simulated pool, and `verify roundtrip` decodes the file back and compares it byte for byte.

Effects and confirmations. Storing, synthesising, sequencing and deleting have effects. A delete needs `confirm physical_key` written on the same statement, and `synthesise` and `sequence` need `confirm hardware`. Wrapping an operation in a function does not hide it:

```
fn purge() returns Void {
    delete "old_archive.bin" from archive
}
let result: Void = purge()
```

Listing 2: A delete inside a function, without its confirmation.

The checker rejects Listing 2 twice, once at the delete and once at the call, which is Destructive because the function is. Adding the confirmation to the delete, where the effect originates, fixes both. `nucle explain` prints each top-level declaration's effect before anything runs.

Effect ceilings. Higher-order functions take closures. A parameter's type can carry a ceiling, and every closure bound into that slot must fit it:

```
fn archive_with_cleanup_policy(cleanup: Fn() -> Void confirm physical_key) returns
    Void {
    let x: Void = cleanup()
}

fn run_cleanup() returns Void {
    let purge_expired: Fn() -> Void confirm physical_key = fn() -> Void {
        delete "sample_a.txt" from archive confirm physical_key
    }
    let result: Void = archive_with_cleanup_policy(purge_expired)
}
```

Listing 3: A closure parameter with a ceiling (`effect_annotated_closure.nsl`, shortened).

Inside `archive_with_cleanup_policy`, the call `cleanup()` is trusted to be Destructive and confirmed without seeing the closure, because each closure that can reach the parameter was checked where it was bound. A closure that stored as well as deleted would be rejected there.

Results, retention and real reads. Operations that can fail at run time return `Result`, which `?` and `match` take apart. Listing 4 asks for ten thousand years of readability, which a pool kept in bone at 25 °C cannot give, and falls back to an encapsulated pool at −20 °C. The check runs against the strands actually written, using the published decay kinetics of the pool's storage medium.

```
pool shelf: DnaPool { codec: Ternary, redundancy: 3x, profile: Twist, storage: bone
    at 25C }
pool capsule: DnaPool { codec: YinYang, redundancy: 3x, profile: Twist, storage:
    encapsulated at -20C }

fn archive_for(target: File) returns Result<DnaFile, Str> {
    let attempt: Result<DnaFile, Str> = store target into shelf { retain: 10000y }
    let saved: DnaFile = match attempt {
        Ok(file) => file,
        Err(reason) => (store target into capsule { retain: 10000y })?
    }
}
```

Listing 4: A retention requirement with a fallback (`retention_policy.nsl`, shortened).

Finally, `decode reads` takes real sequencing reads and the designed references, decodes them with and without the dataset's cluster labels, and stops the program if recovery falls short:

```
decode reads "nucle_ecc/tests/fixtures/cnr/Clusters.txt"
    against "nucle_ecc/tests/fixtures/cnr/Centers.txt" {
    blind,
    expect recovery > 90%
}
```

Listing 5: Decoding real MinION reads (`real_reads.nsl`, with paths from the repository root).

The language also has functions generic over a pool's profile (`fn recover_from<P>(source: Pool<P, 0.35%>)`), user enums with exhaustive matching, package imports, `test` blocks with `assert`, and compile-time `if` and `for`.

4 Types and effects

4.1 Pools typed by their channel

A pool's type is `Pool<`P`, `r`>`, where P is a sequencer profile or `Recovered`, and r an error rate in percent. Simulating a pool under profile P gives the profile's default rate (0.35% for Illumina, 5% for Nanopore, 0.03% for Twist and 0.05% for IDT). `consensus_vote` at coverage $c \geq 2$ divides the rate by c^2, a deliberately simple model of majority voting that the type system treats as a bound, not a prediction. Where branches produce pools, as in the arms of a `match`, the result takes the worse rate: the type promises only the noise every branch can guarantee. A declared rate that disagrees with the inferred one is an error.

Functions can be generic over the profile. A generic body is checked again for each profile it is called with, so a warning that applies only to Nanopore fires at a Nanopore call and not an Illumina one. A recovery expectation such as `expect recovery > 99.9%` is checked against the profile and coverage, and the optimizer raises redundancy that is too low for the chosen profile before anything is stored.

4.2 Effects

An operation may have three effects: Synthesis (Syn), Sequencing (Seq) and Destructive (Des). `nucle explain` reports the worst of a declaration's effects in the order Pure $<$ Seq $<$ Syn $<$ Des. The checker keeps the whole set, because ceilings need it. Table 1 gives the rules. Γ holds the program's functions, the closures `let`-bound so far in the enclosing blocks, and the ceilings of annotated `Fn` parameters.

Form	Effects
`store f into p`	{Syn}
`simulate store`, `retrieve`, `decode reads`, declarations	∅
`delete f from p`	{Des}
`synthesise ...`, `sequence ...`	{Syn}, {Seq}
a pipeline with a `store into` step	{Syn}
a closure literal	∅: defining one runs nothing
$f(a_1, \ldots, a_n)$, f a function or closure in Γ	E(body of f) $\cup$ $E(a_i)$ for each closure a_i passed to an unannotated `Fn` parameter
$p()$, p typed `Fn(...) -> T confirm hardware`	{Syn}
$p()$, p typed `Fn(...) -> T confirm physical_key`	{Des}
$p()$, p an unannotated `Fn` parameter	∅; the call that passed the closure adds its effects
$g()$, g a closure the checker cannot trace	every effect
`match`, `if`	the union over every arm or branch, taken or not
a block $d_1 \ldots d_n$	$\bigcup_i E(d_i)$, with `let` bindings of closures, aliases and annotated types entering Γ

Table 1: Effect rules. A recursive call contributes ∅ to its own strongly connected component, which gives the least fixed point: a recursive function that deletes is still Destructive.

Confirmation. Every operation that needs a confirmation carries it on its own statement. A call is confirmed when every operation its effects come from is. A `store` needs no confirmation; the checker warns about stores that are not simulated.

Ceilings. Binding a closure c into a slot with ceiling k, by passing it to an annotated parameter or `let`-binding it with an annotated type, requires $E(c) \subseteq \mathit{allowed}(k)$ and requires every operation in c to be confirmed. Here $\mathit{allowed}(\texttt{hardware}) = \{\mathsf{Syn}, \mathsf{Seq}\}$ and $\mathit{allowed}(\texttt{physical_key}) = \{\mathsf{Des}\}$. Comparing the whole set matters: a closure that stores and deletes has worst effect Destructive, which fits `confirm physical_key`, but its store does not.

Closures the checker cannot see into. A closure returned by a function, or taken out of an enum, is a value whose body the effect walk never meets. A call through one has every effect, so it fits no ceiling unless the `Fn` type it comes through declares one, which the checker enforced where the closure was created.

Parameters without a ceiling. Inside its function, a call through an unannotated `Fn` parameter counts as ∅, because each call site adds the effects of the closure it passes. That is right for reporting and wrong under a ceiling: a function that passes such a parameter on to a `confirm hardware` slot runs whatever closure its own caller chose under that ceiling. So where a ceiling is checked, a call through an unannotated parameter has every effect.

5 Implementation

NucleScript is implemented in the NucleOS workspace: 14 Rust crates and about 54,000 lines of Rust, of which the compiler crate `nucle_lang` is about 17,000. A program goes through a lexer and parser, a type and effect checker that also resolves compile-time `if` and `for`, a mid-level representation, an optimizer that raises redundancy for the chosen profile and coverage, and either a simulation backend or an interpreter that runs the operations against the NucleOS virtual file system. The effect rules of Table 1 live in `effects.rs` (872 lines) and the ceiling check in `typeck::check_fn_effect_compatibility`.

The layers underneath are:

- **Codecs.** Ternary, DNA Fountain, Yin-Yang and HEDGES, with a self-describing archive header from which the file table can be recovered from a shuffled dump of strands.
- **Error correction.** Reed–Solomon over strand bases, with parity planned from a measured channel and a target failure rate; partial-order consensus followed by a length-aware median polish; and clustering of unlabelled reads by MinHash sketches, locality-sensitive hashing and a banded edit distance, with a merge threshold fitted to the reads.
- **Simulation.** A noise engine with literature profiles and two profiles measured from 1.7 million real Illumina read pairs (Section 8), plus Arrhenius decay for three storage media.
- **Hardware boundary.** `nucle hardware order` turns a program's stores into an oligo-pool order sheet for Twist or IDT, checked against the limits each vendor publishes; adapters for vendor APIs exist but no order has been placed.
- **Tooling.** A language server, a VS Code extension, a formatter, a test runner, documentation generation, and a playground compiled to WebAssembly.

The workspace has 952 tests. NucleScript 1.0 was declared on 7 November 2026 with a stability promise: every 1.x release accepts and runs what 1.0 accepts, with the same effects and confirmations, except for soundness fixes, each of which is logged. A frozen corpus of the 33 example programs that shipped with 1.0 records, for each, whether it is accepted, its diagnostic codes, and each declaration's effect; every release must reproduce it. The three soundness fixes of Section 6 changed none of the 33.

6 Checking the checker

The checker makes four promises about any program it accepts:

1. **Confirmation.** No operation runs without the confirmation it needs.
2. **Effect reports.** No top-level declaration performs an operation with a worse effect than `nucle explain` reports.
3. **Effect ceilings.** A closure bound into a `confirm hardware` slot never deletes while it runs, and one bound into a `confirm physical_key` slot never stores, synthesises or sequences.
4. **Value shapes.** Every value bound, passed or returned has a shape its declared type allows, and `?` and `match` only meet the values they are typed for.

The monitor. While a program runs, the interpreter records every store and delete that reaches the virtual file system, with its effect, whether its source carried its confirmation, and which top-level declaration was running. It records a fault when a value fails its declared type, including a declared pool whose profile is not the one a `Pool<Profile>` names. A closure takes on the ceiling of every slot it is bound into, the interpreter keeps the ceilings of the closure calls in progress, and an operation that exceeds one is a fault. A test then checks each trace against the four promises.

The generator. Programs come from the shipped examples (30 of the 31 compile, and they run 48 stores and deletes) and from a generator in the style of QuickCheck [7] and Csmith [30]. It writes functions, closures held in `let`s, higher-order functions with and without ceilings, closures passed inline or by name, closures returned from functions and passed on under second names, functions generic over a profile, functions that recurse until the call-depth limit stops

	Defect	Found by
1	Pipelines that store were reported Pure.	monitor
2	A closure bound inside a function body was invisible to the effect walk, so a function that deleted through its own closure was Pure.	monitor
3	Through defect 2, a delete could pass a `confirm hardware` ceiling.	monitor
4	A store could pass a `confirm physical_key` ceiling when the same closure also deleted, because ceilings compared only the worst effect.	monitor
5	A `Void` function returned its last binding at run time.	monitor
6	A call through a closure returned from a function counted as Pure and fit every ceiling.	calculus, generator
7	At top level, `let again: Fn() -> Void = wipe` was never evaluated, so `again()` found nothing to call.	calculus, generator
8	A function could pass its own unannotated `Fn` parameter into a `confirm hardware` slot, and a caller could then give it a closure that deletes.	calculus, generator

Table 2: Defects in the checker found by the runtime monitor, by formalising the rules, and by the generator extended afterwards. All are fixed.

them, functions that forward a closure parameter to a function with a ceiling, `Result` functions using `?` and `match`, pipelines, and stores with retention requirements. About one delete in twenty-five drops its confirmation and about one closure in twenty is given a ceiling it does not fit, so rejections are exercised too.

Results. In CI, 300 generated programs run on every push: 215 are accepted, and they run 2,871 stores and 313 deletes. A run of 5,000 programs on 8 November 2026 accepted 3,540, ran 48,086 stores and 4,900 deletes, and broke no promise. The reports are close to exact: of the 45,903 top-level declarations that ran an operation, 131 were reported with a worse effect than they performed, mostly calls through a closure the checker cannot trace. The rules allow that, since every arm counts whether it runs or not and a run that stops early never reaches its later operations.

What it found. The monitor's first run failed on five shipped examples and 159 of the 300 generated programs. Table 2 lists the eight defects found since. The last three came while writing the rules down for the proof of Section 7: the calculus made it plain that a closure can leave the scope the effect walk watches, and its ceiling rule had to say what a parameter without a ceiling may do inside one. The generator was then extended to write such programs. On the code before those fixes it flagged 70 of its 300 programs for the first two, and one for the third. Each defect is pinned by a regression test that fails on the code before its fix.

7 A machine-checked core

Testing shows that the programs run so far kept their promises. To show that every accepted program does, the effect and confirmation rules were written down in Lean 4 [9] for a core of the language, and the first three promises proved for it. The development is one file of 651 lines with no library beyond Lean's core; the three main theorems use no axiom but propositional extensionality, and CI rebuilds the proof on every push and fails if any step rests on `sorry`.

7.1 The core calculus

Effects are the three base effects; effect sets are predicates on them. Statements are:

$$\begin{aligned} s \;::=\;& \mathsf{op}(o) \mid s_1; s_2 \mid s_1 \oplus s_2 \mid \mathsf{let}\ x{:}k = \mathsf{fn}\{s_b\};\ s \mid \mathsf{let}\ x = y;\ s \\ \mid\;& \mathsf{let}\ x{:}k = \mathsf{opaque}\{s_b\};\ s \mid x() \mid f(x_1, \ldots, x_n) \end{aligned}$$

An operation o has an optional effect and a flag saying whether its confirmation was written. $s_1 \oplus s_2$ is a branch (`if` or `match`) of which either side may run. k is an optional ceiling. $\mathsf{opaque}\{s_b\}$ is a closure the rules cannot see into, such as one returned by a function: s_b is what it does, which the effect rule for calls never looks at. Named functions take closure arguments, each parameter with or without a ceiling, and capture nothing. Closures take no arguments of their own. Pools, files and `Result` add no effects of their own and are left out.

Semantics. A closure value $\mathsf{clo}(s_b, \rho, ks)$ holds its body, its captured environment and the ceilings of the slots it was bound into. The big-step judgment $\rho \vdash_{cs} s \Downarrow t$ runs s under the ceilings cs of the closure calls in progress and produces the list t of events it performed; each event records an effect, whether its confirmation was written, and cs, as the monitor's trace does. The rules that matter are the closure call, which runs the body under $ks +\!\!+ cs$, and argument binding, which adds a parameter's ceiling to the value's ks:

$$\frac{\rho(x) = \mathsf{clo}(s_b, \rho', ks) \qquad \rho' \vdash_{ks +\!\!+ cs} s_b \Downarrow t}{\rho \vdash_{cs} x() \Downarrow t}$$

$$\frac{P(f) = (\bar{p}{:}\bar{k},\ s_f) \qquad \rho_f = [p_i \mapsto \rho(x_i) \text{ under } k_i] \qquad \rho_f \vdash_{cs} s_f \Downarrow t}{\rho \vdash_{cs} f(\bar{x}) \Downarrow t}$$

Rules. The judgment $\Gamma \vdash s : R, U$ gives a statement two things: R, the effects it may have itself, and U, the parameters without a ceiling whose closures it may call. A name in Γ is bound either to $\mathsf{known}(R, U)$, a closure whose effects are known, or to $\mathsf{param}(p)$, a parameter without a ceiling, with $R(\mathsf{param}(p)) = \emptyset$ and $U(\mathsf{param}(p)) = [p]$. R is what `nucle explain` reports; a call adds the effects of the closures it passes for U. A closure fits a ceiling only if it calls no parameter without one:

$$\mathit{fits}(k, R, U) \iff \forall c.\ k = c \Rightarrow U = [\,] \wedge R \subseteq \mathit{allowed}(c)$$

This is the rule defect 8 broke. The rules for closures and calls are:

$$\frac{\Gamma \vdash s_b : R_b, U_b \qquad \mathit{fits}(k, R_b, U_b) \qquad \Gamma, x{:}\mathsf{known}(R_b, U_b) \vdash s : R, U}{\Gamma \vdash \mathsf{let}\ x{:}k = \mathsf{fn}\{s_b\};\ s : R, U}$$

$$\frac{\Gamma(y) = b \qquad \Gamma, x{:}b \vdash s : R, U}{\Gamma \vdash \mathsf{let}\ x = y;\ s : R, U}$$

$$\frac{\Gamma \vdash s_b : R_b, U_b \qquad \mathit{fits}(k, R_b, U_b) \qquad \Gamma, x{:}\mathsf{known}(\mathit{opq}(k), [\,]) \vdash s : R, U}{\Gamma \vdash \mathsf{let}\ x{:}k = \mathsf{opaque}\{s_b\};\ s : R, U} \qquad \frac{\Gamma(x) = b}{\Gamma \vdash x() : R(b), U(b)}$$

$$\frac{P(f) = (\bar{p}{:}\bar{k},\ s_f) \qquad \Gamma \vdash \bar{x} \triangleright \bar{p}{:}\bar{k} : R_a, U_a}{\Gamma \vdash f(\bar{x}) : \Phi(f) \cup R_a,\ U_a}$$

where $\mathit{opq}(\cdot)$ is every effect without a ceiling and $\mathit{allowed}(c)$ with one, which is defect 6's fix. The argument judgment $\Gamma \vdash \bar{x} \triangleright \bar{p}{:}\bar{k} : R_a, U_a$ adds $R(b)$ and $U(b)$ for each argument bound to b that goes to a parameter without a ceiling, and requires $U(b) = [\,]$ and $R(b) \subseteq \mathit{allowed}(c)$ for one with ceiling c. $\Phi(f)$ is the effect of calling f; a program is accepted when each function body's R is within its Φ. The least fixed point the checker computes (Table 1) is one such Φ.

7.2 Theorems

Write $\mathit{eff}(ev)$, $\mathit{conf}(ev)$ and $\mathit{ceil}(ev)$ for an event's effect, confirmation flag and ceilings.

Theorem 1 (Effect reports)**.** *If every function of P is accepted, $\cdot \vdash s : R, U$ and $[\,] \vdash_{[\,]} s \Downarrow t$, then $\mathit{eff}(ev) \in R$ for every $ev \in t$.*

Theorem 2 (Effect ceilings)**.** *Under the same assumptions, $\mathit{eff}(ev) \in \mathit{allowed}(c)$ for every $ev \in t$ and every $c \in \mathit{ceil}(ev)$.*

Theorem 3 (Confirmation)**.** *If every operation in s and in the bodies of P carries the confirmation it needs and $[\,] \vdash_{[\,]} s \Downarrow t$, then $\mathit{conf}(ev)$ holds for every $ev \in t$.*

The proof needs an invariant the rules do not state: a closure value does at most what the binding that holds it says, given what the enclosing function's parameters hold. With σ mapping each parameter to the effects its closures may have, a binding means $[\![b]\!]_\sigma = R(b) \cup \bigcup_{p \in U(b)} \sigma(p)$, and $\mathit{ValOk}_B(\mathsf{clo}(s_b, \rho', ks))$ holds when, for some Γ' and σ' with $\Gamma' \vdash s_b : R, U$ and ρ' matching Γ' under σ', $[\![\mathsf{known}(R, U)]\!]_{\sigma'}$ is within B and within $\mathit{allowed}(c)$ for each $c \in ks$. An environment matches a context when each name's value is *ValOk* for its binding's meaning.

Lemma 1. *If $\rho \vdash_{cs} s \Downarrow t$, $\Gamma \vdash s : R, U$ and ρ matches Γ under σ, then for every $ev \in t$, $\mathit{eff}(ev) \in [\![\mathsf{known}(R, U)]\!]_\sigma$, and every $c \in \mathit{ceil}(ev)$ is in cs or allows $\mathit{eff}(ev)$.*

The proof is by induction on the evaluation. In the call to a named function, the callee's environment matches its parameter context under the constant $\sigma_f = \lambda p.\ R_a \cup \bigcup_{q \in U_a} \sigma(q)$: each parameter without a ceiling is bounded by everything the call passes, which is coarse but enough, and each parameter with a ceiling holds a value that took the ceiling on. The theorems follow with $\sigma = \lambda p.\ \emptyset$, since a whole program starts with no parameters and no ceilings in force.

7.3 What the proof does not cover

The Lean rules were written by hand from `effects.rs` and `typeck.rs`, and nothing ties the two together; a slip in the Rust that the Lean rules do not share is still caught only by the monitor and the tests. The core leaves out closures with arguments of their own, generics, `Result` and `?`, enums except as a source of opaque closures, and value shapes, so the fourth promise rests on testing alone.

8 Decoding real reads

Simulated noise only tests a model of the channel, so NucleOS also decodes public sequencing data.

MinION reads. The Clustered Nanopore Reads dataset [26] has 10,000 Twist-synthesised strands of 110 nt and 269,709 Oxford Nanopore MinION reads. On its first 2,000 clusters, at 5.77% raw per-base error, the partial-order vote alone recovers 81.8% of strands exactly. Its failures share one shape: a homopolymer called one base short or long, or junk bases at an end, shared by a majority of the cluster's reads. A median-string polish that knows the strand length the codec wrote raises exact recovery to 94.7% and lowers residual error from 0.338% to 0.100%. With the labels discarded, clustering at an edit distance of 15% of the length gives 2,096 clusters, all pure, and recovers 94.70%, against 94.65% with the true labels.

Illumina oligo pools. Gimpel et al. [12] synthesised one benchmark pool of 11,293 strands of 129 nt under a high- and a low-fidelity scenario and sequenced both on an Illumina iSeq 100 (ENA study ERP173549). Read 1 of each pair was assigned to its reference by 12-mer voting and trimmed of adapter, keeping up to 30 reads per reference. Table 3 gives the results. The nanopore merge threshold lost five points here: 15% edit distance joins distinct strands when reads are 99.8% accurate, and many references in this pool are tandem repeats a few bases

Decoder	High fidelity	Low fidelity
Rust examples: 306,408 and 292,119 reads, 0.16% and 1.70% raw error		
Partial-order vote	99.74%	95.41%
Vote and length-aware median polish	99.76%	96.81%
Labels discarded, nanopore threshold (0.15)	93.93%	92.16%
Labels discarded, fitted threshold	99.71%	95.79%
NucleScript program: 306,715 and 292,942 reads, 0.22% and 1.85% raw error		
Labels kept	99.73%	96.94%
Labels discarded, fitted threshold	99.73%	95.82%

Table 3: Strands recovered exactly from the two ERP173549 pools. The NucleScript rows come from `decode reads` on pools a separate script rebuilds from ENA; its adapter trimming keeps slightly more reads.

apart. Clustering now measures the noise first, from the distance between sampled reads and their nearest look-alikes, and sets the threshold from it: 0.03 for the clean pool, 0.05 for the noisy one and 0.15 for MinION.

The NucleScript rows come from a program of two `decode reads` statements, run on pools that a standard-library Python script rebuilds from the published reads and references. Its trimming differs slightly: in the low-fidelity pool only about two reads in three carry an exact copy of the adapter, which was synthesised with the oligo and carries its errors, so the script also finds the adapter by part of its sequence and otherwise cuts at 129 nt. Every recovery figure is within 0.13 points of the Rust examples'. Both pools took 20 minutes with a release build.

Measured channels. The same reads give the simulator two measured profiles (Figure 1). An error that both mates of a read pair share is in the molecule; the rest was added by the sequencer. In the high-fidelity pool the reads miss 0.075% of bases by substitution, 0.0056% by insertion and 0.028% by deletion, and 91% of reads are error-free; in the low-fidelity pool the rates are 0.98%, 0.10% and 0.43%, and 19.3% of reads are error-free. Coverage per oligo is log-normal with σ of 0.77 and 1.14 in log space. The measurement also showed that the Illumina profile overstated the sequencer's own insertions and deletions 5 to 40 times; it now uses 2×10^{-5}.

9 Related work

Languages for designing DNA. Cello compiles Verilog into genetic circuits [18], Eugene constrains how biological parts compose [3], GSL describes genome edits [29], and SBOL is a data standard for such designs [11]. Their values are biological parts; NucleScript's are stored files and pools read through noisy machines. They meet at the strand: NucleScript's sequence literals are checked for GC content, homopolymers and hairpins, as such tools also do.

Languages for laboratory work. BioCoder writes protocols as programs [2]. Puddle is an operating system and runtime for digital microfluidics that corrects droplet errors as a program runs [28]. Puddle is the nearest relative in spirit, a full stack over physical operations that can fail, and unlike NucleScript it drives real hardware. NucleScript checks before anything runs instead: Puddle handles failure dynamically and NucleScript plans for it statically.

DNA storage systems. Bornholt et al. designed an archival system with random access through PCR primers [4]; Organick et al. stored and randomly accessed about 200 MB [19]; Takahashi et al. automated the whole cycle in one device [27]. Chamaeleo [21], MESA [24], DNA-Storalator [5] and the DNA Storage Toolkit [25] are libraries and simulators. NucleOS

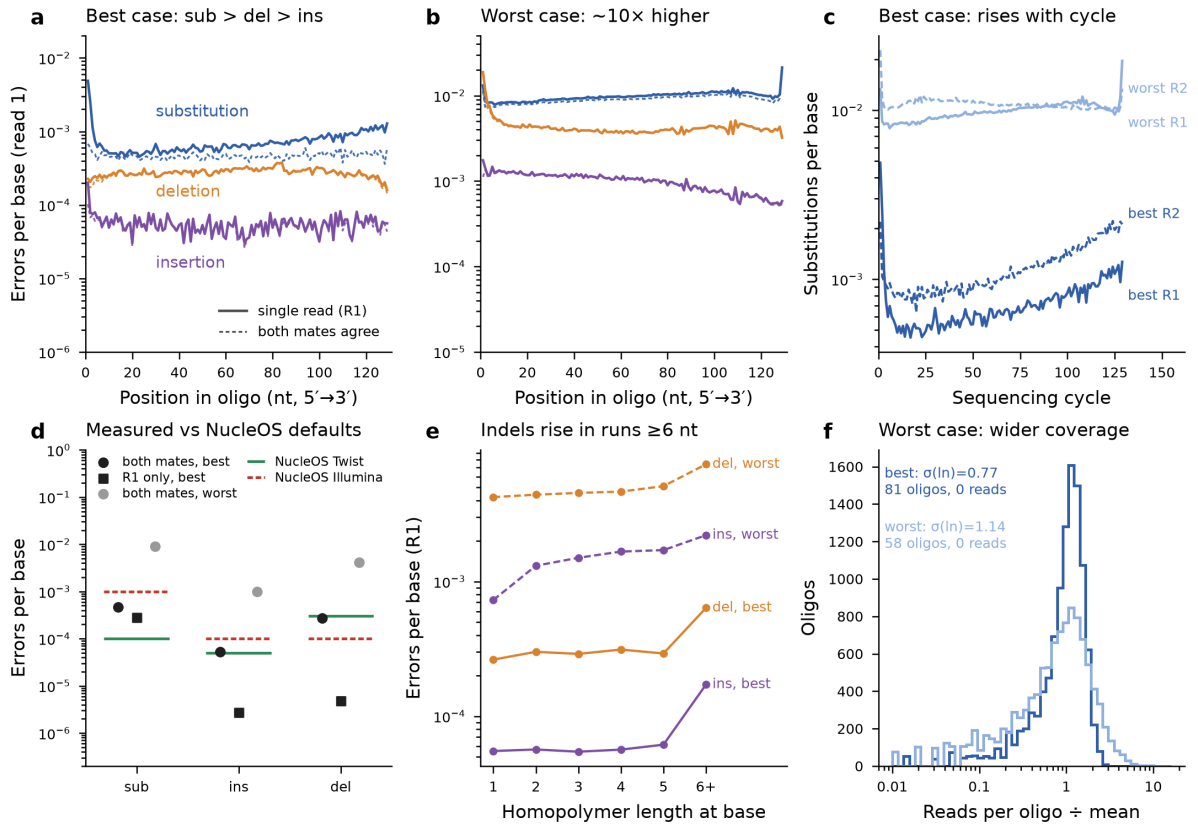


Figure 1: The two ERP173549 pools. (a, b) Read-1 error by position, single read (solid) and both mates agreeing (dashed). (c) Substitutions by sequencing cycle. (d) Molecule-level and sequencer rates against the earlier defaults. (e) Indels by homopolymer length. (f) Reads per oligo over the mean.

covers similar ground; what they lack is a language over those parts with types and effects a compiler checks. The systems papers ran real synthesis and sequencing, which NucleOS has not.

Effects and physical quantities in types. NucleScript's effects follow the effect systems of Lucassen and Gifford [17]: a function's effect is the join of its body's. Koka carries effects in function types [16], as NucleScript's ceilings do. An error rate in a pool's type is closest to units of measure [15], a physical quantity the checker carries.

Testing and proving checkers. The monitor and generator follow QuickCheck [7] and Csmith [30]. The proof is an induction over a big-step semantics with a semantic invariant on closure values, mechanised in Lean [9]. The method is standard; what the proof adds is the rules it covers, in particular closures the checker cannot trace and parameters without a ceiling, both of which the implementation got wrong until the rules were written down.

10 Limitations

- **No wet-lab run.** No NucleScript program has driven a real synthesis or sequencing run. The order sheets have not been submitted. The real-read results use public datasets made by others.
- **IDT's template.** IDT's oPools page takes Excel files only and, without an account, shows neither its template nor its column names, so NucleOS writes a CSV to paste into it. The length and pool size limits it checks matched the page on 8 November 2026.

- **The proof covers a core.** See Section 7; the fourth promise and the rest of the language rest on testing.
- **Error-rate types are estimates.** The rates come from published profiles and from the measured channels of one study, and the consensus rule is a simple bound. They say nothing about a lab's own instruments.
- **Synthesis and sequencing share a ceiling**, and `store` needs no confirmation, by design.
- **No archived DOI yet.** The software is described for archiving on Zenodo, but no DOI has been minted.

11 Availability

NucleOS and NucleScript are MIT-licensed at `https://github.com/VyomKulshrestha/Nucle-OS`, and this report describes release v0.1.70. A playground runs in the browser at `https://nuclescript.github.io/playground/`. In the repository:

- `proofs/`: the Lean proof, built with `lake build` on Lean 4.35.0.
- `nucle_lang/tests/soundness.rs` and `soundness_regressions.rs`: the monitor's tests and the eight regressions; `NSL_SOUNDNESS_PROGRAMS` sets the number of generated programs.
- `scripts/erp173549/`: `prepare.py` rebuilds the Illumina pools and `decode.nsl` decodes them. The references are GPL-3.0, so no copy ships with the repository.

References

[1] M. E. Allentoft, M. Collins, D. Harker, et al. The half-life of DNA in bone: measuring decay kinetics in 158 dated fossils. *Proceedings of the Royal Society B*, 279(1748):4724–4733, 2012. doi:10.1098/rspb.2012.1745.

[2] V. Ananthanarayanan and W. Thies. BioCoder: A programming language for standardizing and automating biology protocols. *Journal of Biological Engineering*, 4:13, 2010. doi:10.1186/1754-1611-4-13.

[3] L. Bilitchenko, A. Liu, S. Cheung, et al. Eugene – a domain specific language for specifying and constraining synthetic biological parts, devices, and systems. *PLoS ONE*, 6(4):e18882, 2011. doi:10.1371/journal.pone.0018882.

[4] J. Bornholt, R. Lopez, D. M. Carmean, et al. A DNA-based archival storage system. In *ASPLOS '16*, pages 637–649, 2016. doi:10.1145/2872362.2872397.

[5] G. Chaykin, O. Sabary, N. Furman, et al. DNA-Storalator: a computational simulator for DNA data storage. *BMC Bioinformatics*, 26:204, 2025. doi:10.1186/s12859-025-06222-0.

[6] G. M. Church, Y. Gao, and S. Kosuri. Next-generation digital information storage in DNA. *Science*, 337(6102):1628, 2012. doi:10.1126/science.1226355.

[7] K. Claessen and J. Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. In *ICFP '00*, pages 268–279, 2000. doi:10.1145/351240.351266.

[8] D. Coudy, M. Colotte, A. Luis, S. Tuffet, and J. Bonnet. Long term conservation of DNA at ambient temperature. Implications for DNA data storage. *PLOS ONE*, 16(11):e0259868, 2021. doi:10.1371/journal.pone.0259868.

[9] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, pages 625–635, 2021. doi:10.1007/978-3-030-79876-5_37.

[10] Y. Erlich and D. Zielinski. DNA Fountain enables a robust and efficient storage architecture. *Science*, 355(6328):950–954, 2017. doi:10.1126/science.aaj2038.

[11] M. Galdzicki, K. P. Clancy, E. Oberortner, et al. The Synthetic Biology Open Language (SBOL) provides a community standard for communicating designs in synthetic biology. *Nature Biotechnology*, 32(6):545–550, 2014. doi:10.1038/nbt.2891.

[12] A. L. Gimpel, A. Remschak, W. J. Stark, R. Heckel, and R. N. Grass. Comparison of state-of-the-art error-correction coding for sequence-based DNA data storage. *Nature Communications*, 2026. doi:10.1038/s41467-026-70548-3. Reads: ENA study ERP173549.

[13] N. Goldman, P. Bertone, S. Chen, et al. Towards practical, high-capacity, low-maintenance information storage in synthesized DNA. *Nature*, 494(7435):77–80, 2013. doi:10.1038/nature11875.

[14] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark. Robust chemical preservation of digital information on DNA in silica with error-correcting codes. *Angewandte Chemie International Edition*, 54(8):2552–2555, 2015. doi:10.1002/anie.201411378.

[15] A. J. Kennedy. Relational parametricity and units of measure. In *POPL '97*, pages 442–455, 1997. doi:10.1145/263699.263761.

[16] D. Leijen. Koka: Programming with row polymorphic effect types. *EPTCS*, 153:100–126, 2014. doi:10.4204/EPTCS.153.8.

[17] J. M. Lucassen and D. K. Gifford. Polymorphic effect systems. In *POPL '88*, pages 47–57, 1988. doi:10.1145/73560.73564.

[18] A. A. K. Nielsen, B. S. Der, J. Shin, et al. Genetic circuit design automation. *Science*, 352(6281):aac7341, 2016. doi:10.1126/science.aac7341.

[19] L. Organick, S. D. Ang, Y.-J. Chen, et al. Random access in large-scale DNA data storage. *Nature Biotechnology*, 36(3):242–248, 2018. doi:10.1038/nbt.4079.

[20] Z. Ping, S. Chen, G. Zhou, et al. Towards practical and robust DNA-based data archiving using the yin–yang codec system. *Nature Computational Science*, 2:234–242, 2022. doi:10.1038/s43588-022-00231-2.

[21] Z. Ping, H. Zhang, S. Chen, et al. Chamaeleo: a robust library for DNA storage coding schemes. *bioRxiv*, 2020. doi:10.1101/2020.01.02.892588.

[22] W. H. Press, J. A. Hawkins, S. K. Jones, J. M. Schaub, and I. J. Finkelstein. HEDGES error-correcting code for DNA storage corrects indels and allows sequence constraints. *PNAS*, 117(31):18489–18496, 2020. doi:10.1073/pnas.2004821117.

[23] C. Rashtchian, K. Makarychev, M. Rácz, et al. Clustering billions of reads for DNA data storage. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[24] M. Schwarz, M. Welzel, T. Kabdullayeva, et al. MESA: automated assessment of synthetic DNA fragments and simulation of DNA synthesis, storage, sequencing and PCR errors. *Bioinformatics*, 36(11):3322–3326, 2020. doi:10.1093/bioinformatics/btaa140.

[25] P. Sharma, G. G. Yipeng, B. Gao, et al. DNA Storage Toolkit: A modular end-to-end DNA data storage codec and simulator. In *ISPASS 2024*, pages 144–155, 2024. doi:10.1109/ISPASS61541.2024.00023.

[26] S. R. Srinivasavaradhan, S. Gopi, H. D. Pfister, and S. Yekhanin. Trellis BMA: Coded trace reconstruction on IDS channels for DNA storage. In *IEEE International Symposium on Information Theory (ISIT)*, pages 2453–2458, 2021. doi:10.1109/ISIT45174.2021.9517821.

[27] C. N. Takahashi, B. H. Nguyen, K. Strauss, and L. Ceze. Demonstration of end-to-end automation of DNA data storage. *Scientific Reports*, 9:4998, 2019. doi:10.1038/s41598-019-41228-8.

[28] M. Willsey, A. P. Stephenson, C. Takahashi, et al. Puddle: A dynamic, error-correcting, full-stack microfluidics platform. In *ASPLOS '19*, pages 183–197, 2019. doi:10.1145/3297858.3304027.

[29] E. H. Wilson, S. Sagawa, J. W. Weis, et al. Genotype Specification Language. *ACS Synthetic Biology*, 5(6):471–478, 2016. doi:10.1021/acssynbio.5b00194.

[30] X. Yang, Y. Chen, E. Eide, and J. Regehr. Finding and understanding bugs in C compilers. In *PLDI '11*, pages 283–294, 2011. doi:10.1145/1993498.1993532.